

Week 10 Tutorial

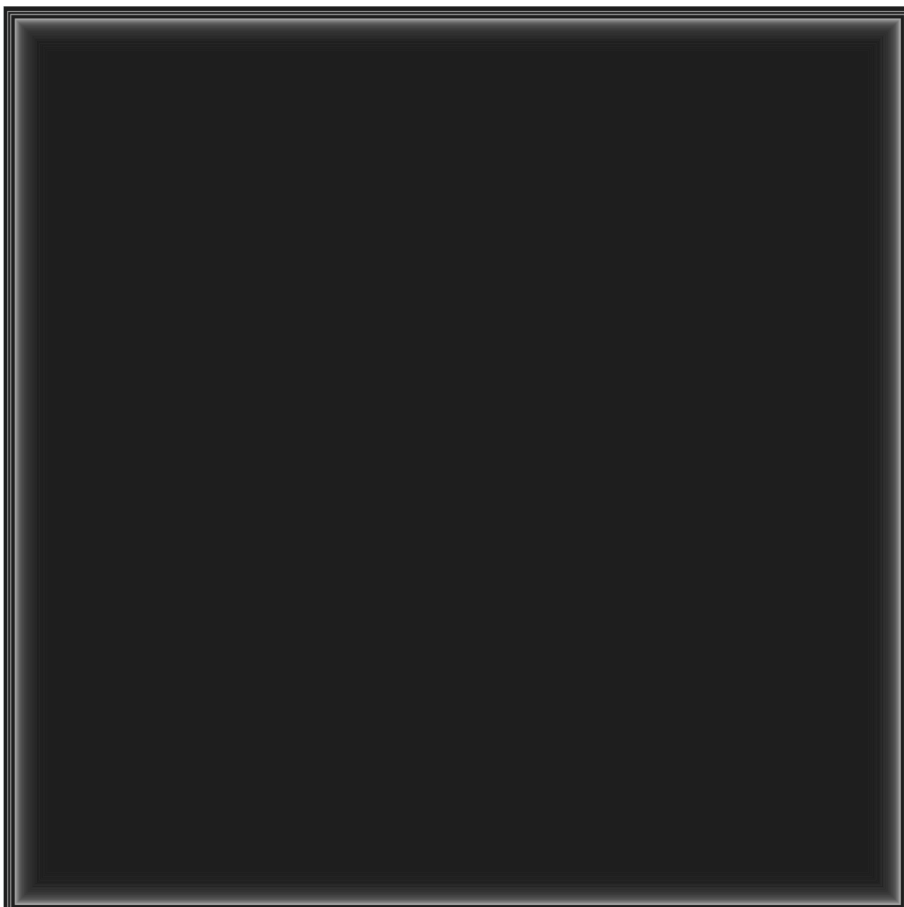
Welcome to this week's tutorial! Today we will be covering:

- Easing
- Transformations: translate, rotate and scale

Easing

Easing is a technique for smoothing the transition between two different states. It helps to make animation look more natural. There are a number of different types of smoothing, which you can learn more about here: <http://easings.net/> ➔ [\(http://easings.net/\)](http://easings.net/). Today we will be implementing a simple smoothing technique that uses the `lerp()` function (lerp is short for linear interpolation).

Part 1: Follow the mouse



Objective:

In this first part we will draw a square that changes size when the mouse moves up and down the canvas. The size of the square will be defined by the distance between the mouse position in the vertical axis and the centre of the screen. Instead of directly changing the size of the square, it will use easing to gradually reach the size, creating a smooth animation.

Challenge:

- We need to set a target size for the square based on the mouse's y-position.
- We need to update the size of square each frame so it smoothly transitions towards that target size.

Solution:

Rather than updating the size of the square directly based on the mouse's y-position, we use `lerp()` to make the movement smooth. To do this, we use an intermediate size called `squareSize`. This intermediate size is calculated by linearly interpolating between the current square size and the target square size. We linearly interpolate by a fixed amount (0.05) each frame. Each frame the difference between the current and target size reduces. This results in the interpolated amount also reducing in size over time, giving the impression that the movement is slowing over time.

Implementation:

Note that for p5.js to detect a change in the mouse position, the browser needs to have focus (i.e. try clicking on the canvas if the square is not transitioning correctly).

```
// We need variables to store the current square size between frames
// as well as the target square size based on the current y-position
// of the mouse
let squareSize;
let targetSquareSize;

// Set the easing value to a constant
// Try changing this value to see how it affects the transition
const easing = 0.05;

function setup() {
  createCanvas(windowWidth, windowHeight);
  background(230);

  // We want the squares drawn to fade over time, so use a semi-transparent fill
  fill(20, 20);
  stroke(170);

  // We want to draw the square in the middle of the canvas,
  // which is easier to achieve using rectMode(CENTER)
  rectMode(CENTER);

  // Start with a random square size as the nominated current size
  squareSize = random(height);
}

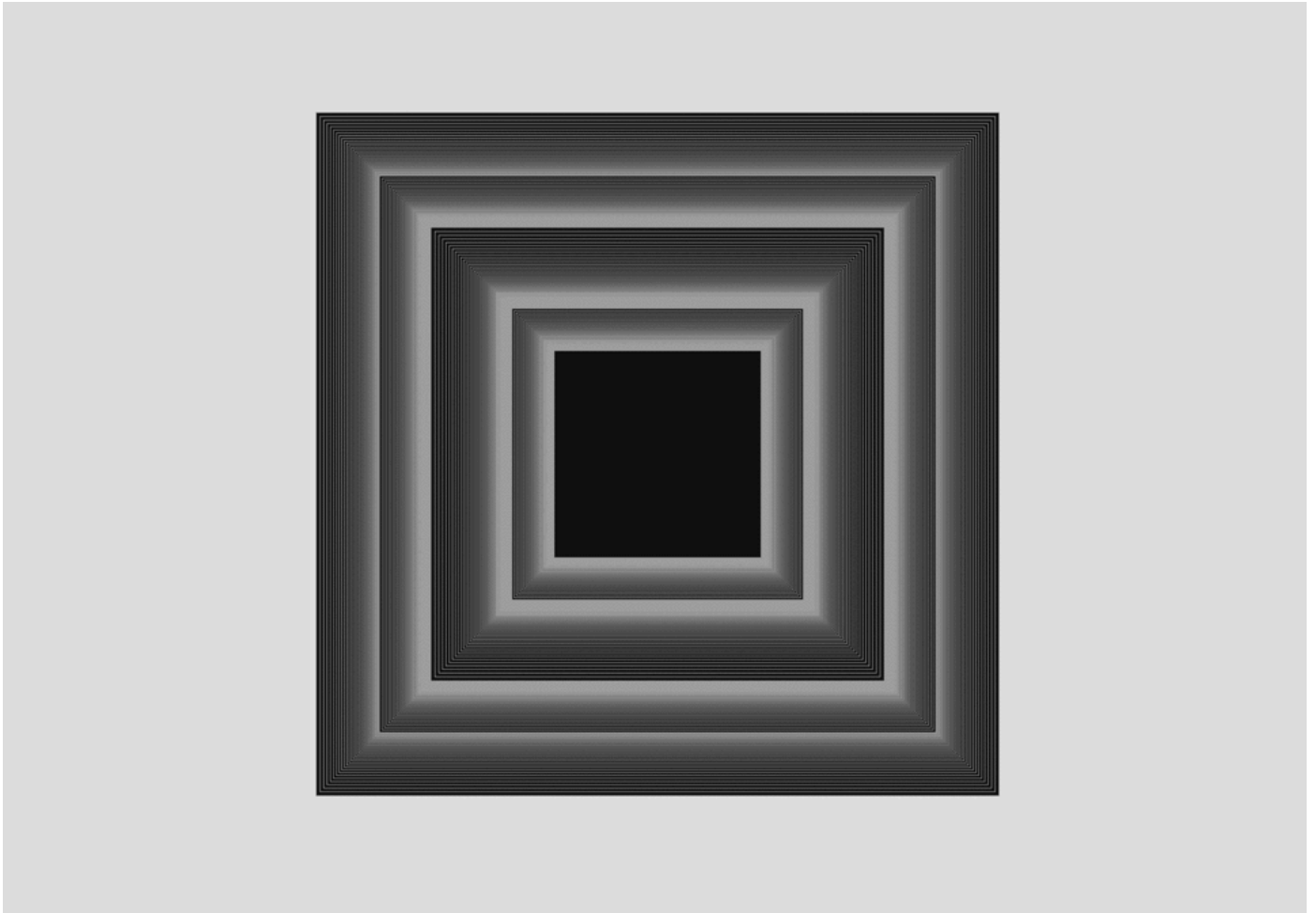
function draw() {
  // Try uncomment the following line to see what this looks like without
  // accumulating the squares.
  //background(230)

  // The target square size is based on the y-position of the mouse
  // By using the abs() function, we calculate the same size of the square
  // regardless whether the mouse cursor is above or below the canvas midpoint.
  targetSquareSize = 2 * abs(mouseY - height / 2);

  // The new square size is calculated by linearly interpolating between the
  // last square size drawn and the target square size as determined using the
  // mouse's y-position.
  squareSize = lerp(squareSize, targetSquareSize, easing);
```

```
// Draw the square using the updated size in the middle of the canvas  
square(width / 2, height / 2, squareSize);  
}
```

Part 2: Change periodically



Objective:

Now that we have a smooth animation for our square, let's change the trigger to be automated.

Challenge:

We want the square size to be updated every 120 frames (or 2 seconds assuming the sketch is running at 60 frames per second).

Solution:

We remove the code related to `mouseY` and replace it with a conditional statement that checks if the `frameCount` is a multiple of 120. To do this, we use the remainder (`%`) operator. The remainder will calculate to 0 every 120 frames and it's at that point that we calculate a new target square size.

Implementation:

```
// We need variables to store the current square size between frames  
// as well as the target square size based on a random value calculated  
// every 120 frames
```

```

let squareSize;
let targetSquareSize;

// Set the easing value to a constant
// Try changing this value to see how it affects the transition
const easing = 0.05;

function setup() {
  createCanvas(windowWidth, windowHeight);
  background(230);

  // We want the squares drawn to fade over time, so use a semi-transparent fill
  fill(20, 20);
  stroke(170);

  // We want to draw the square in the middle of the canvas,
  // which is easier to achieve using rectMode(CENTER)
  rectMode(CENTER);

  // Start with a random square size as the nominated current size
  squareSize = random(height);

  // The initial target square size is set to half the current square size
  targetSquareSize = squareSize / 2;
}

function draw() {
  // Every 120 frames (or 2 seconds assuming the sketch is running at
  // 60 frames per second) a new target size is calculated at random
  if (frameCount % 120 == 0) {
    targetSquareSize = random(height / 2);
  }

  // The new square size is calculated by linearly interpolating between the
  // last square size drawn and the target square size that gets updated
  // every 120 frames
  squareSize = lerp(squareSize, targetSquareSize, easing);

  // Draw the square using the updated size in the middle of the canvas
  square(width / 2, height / 2, squareSize);
}

```

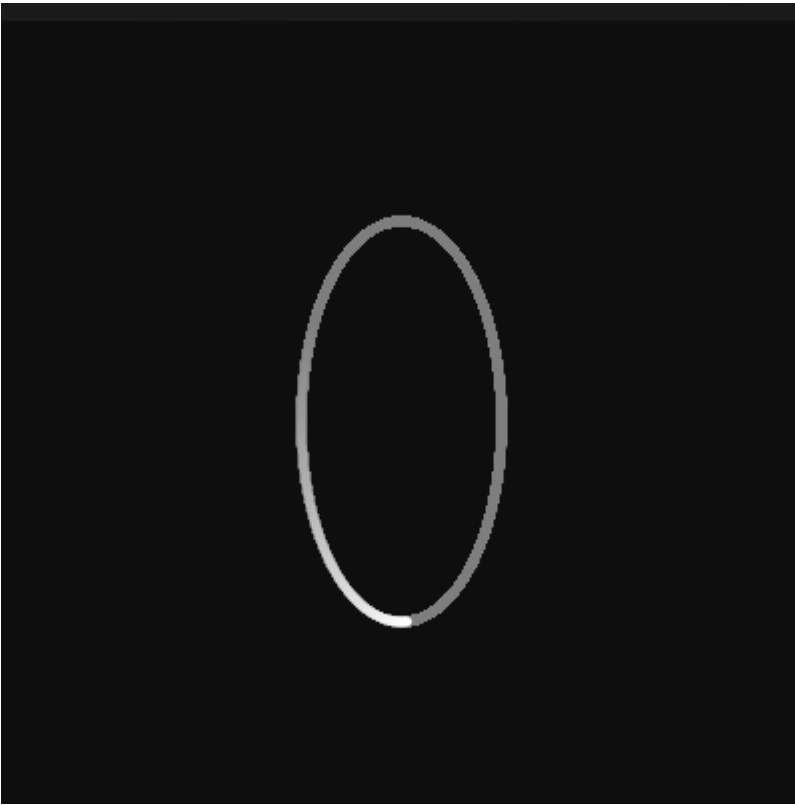
Transformations

Transformations allow us to manipulate the position, size and angle of the shapes we draw on the canvas. The main methods we use to perform transformations are `translate()`, `rotate()` and `scale()`. When we perform a transformation in p5.js we are actually manipulating something called the *model matrix*. When working in 2D, we can just think about transformation as manipulation of the canvas that the shapes are drawn on. `translate()` will move the canvas up, down, left and right. `rotate()` will rotate the canvas around its origin (i.e. 0,0). And `scale()` will zoom the canvas in and out.

It is important to know that the order in which you perform the transformations matters. This is because each transformation affects the next one. Remember, you are transforming the coordinate system rather than the shape itself. Whilst transformations can be performed in any order, the order which usually feels the most intuitive is translate, rotate, then scale.

You can read more about transformations in p5.js here: [Coordinates and Transformations](https://archive.p5js.org/learn/getting-started-in-webgl-coords-and-transform.html) 
[\(https://archive.p5js.org/learn/getting-started-in-webgl-coords-and-transform.html\)](https://archive.p5js.org/learn/getting-started-in-webgl-coords-and-transform.html)

Part 1: Trace an ellipse



Objective:

We want to draw a dot that moves in an elliptical orbit, creating an ellipse on the canvas. Instead of moving the position of the ellipse using its draw coordinates, we will use `translate()` and `rotate()` to achieve this.

Challenge:

We need a way to move the position of what we draw each frame. To do this we can use the `translate()` function, which moves the co-ordinate system and in effect is moving the origin position of the canvas.

Solution:

- Before we draw the circle, move the origin position by calling `translate()` twice. The first time `translate()` is called, the origin position is moved to the centre of the canvas. The second time `translate()` is called, the origin position is moved in an elliptical pattern using trigonometry (see: [Polar coordinate system](https://en.wikipedia.org/wiki/Polar_coordinate_system) (https://en.wikipedia.org/wiki/Polar_coordinate_system)).
- Now that the canvas has been moved, we simply draw the dot at the origin position (0,0) using the `circle()` function. Due to the preceding calls to `translate()`, the circle is drawn at the correct position along an elliptical orbit.

Implementation:

```
function setup() {  
  createCanvas(400, 400);  
  background(0);  
  angleMode(DEGREES);  
}
```

```
function draw() {
  // Set the background to black with an alpha value of 1,
  // so that the dot drawn has a visible trail
  background(0, 1);

  // Move the origin position to the centre of the canvas
  translate(width / 2, height / 2);

  // Calculate translateX and translateY,
  // so that the dot follows an elliptical pattern
  let translateX = sin(frameCount) * width / 8;
  let translateY = cos(frameCount) * height / 4;

  // Move the origin position again using translateX and translateY
  translate(translateX, translateY);

  // Draw the dot by drawing a circle with diameter 5
  // at the current origin (0,0) position, which has
  // been manipulated by the two calls to translate() above
  noStroke();
  fill(255);
  circle(0, 0, 5);
}
```

Part 2: Add complexity



Objective:

Now that we have our dot drawing an elliptical orbit, we want to vary it to create a more complex shape.

Challenge:

Is there a way we can add more complexity by adding a single additional transformation? If we add a rotation between the two translations, we could change the nature of the shape being

drawn.

Solution:

By calling `rotate()` we are effectively rotating the canvas around the origin. And if we rotate the canvas proportional to the x-y dimensions of the ellipse, we can create a harmonic pattern.

Implementation:

Try to vary the value `1600` in the call to `rotate()` to see what patterns can be produced.

```
function setup() {
  createCanvas(400, 400);
  background(0);
  angleMode(DEGREES);
}

function draw() {
  // Set the background to black with an alpha value of 1,
  // so that the dot drawn has a visible trail
  background(0, 1);

  // Move the origin position to the centre of the canvas
  translate(width / 2, width / 2);

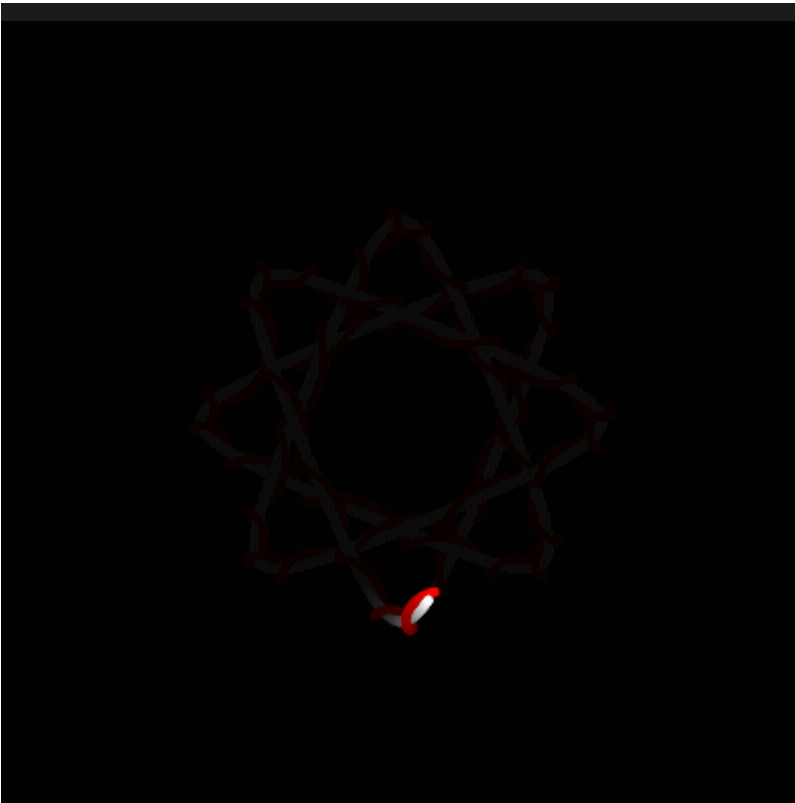
  // Slowly rotate the co-ordinate system
  // (effectively rotating the canvas around the origin)
  // Try changing the number 1600 to see what patterns you can make
  rotate(frameCount * width / 1600);

  // Calculate translateX and translateY,
  // so that the dot follows an elliptical pattern
  let translateX = sin(frameCount) * width / 8;
  let translateY = cos(frameCount) * height / 4;

  // Move the origin position again using translateX and translateY
  translate(translateX, translateY);

  // Draw the dot by drawing a circle with diameter 5
  // at the current origin (0,0) position, which has
  // been manipulated by the two calls to translate() above
  noStroke();
  fill(255);
  circle(0, 0, 5);
}
```

Part 3: Chase the dot



Objective:

We can add even more complexity to our design by adding additional elements. Draw another dot that orbits around the first one as it moves around the canvas.

Challenge:

Because transformations are cumulative, we do not need to perform the same transformations again when drawing our second dot. Instead, we only need to consider how the transformations of our second dot differ from the first, and perform those specific transformations before drawing our second dot.

Solution:

Seeing as we would like the second dot to orbit the first, the additional transformation required is one which performs an orbit around the origin position. We use the same algorithm, $x = r \cdot \cos(\theta)$ and $y = r \cdot \sin(\theta)$, that we used to move the first dot before we draw the second dot.

Implementation:

```
function setup() {  
  createCanvas(400, 400);  
  background(0);  
  angleMode(DEGREES);  
}  
  
function draw() {  
  // Set the background to black with an alpha value of 1,  
  // so that the dot drawn has a visible trail  
  background(0, 1);  
  
  // Move the origin position to the centre of the canvas  
  translate(width / 2, width / 2);
```



```
// Slowly rotate the co-ordinate system
// (effectively rotating the canvas around the origin)
// Try changing the number 1600 to see what patterns you can make
rotate(frameCount * width / 1600);

// Calculate translateX and translateY,
// so that the dot follows an elliptical pattern
let translateX = sin(frameCount) * width / 8;
let translateY = cos(frameCount) * height / 4;

// Move the origin position again using translateX and translateY
translate(translateX, translateY);

// Draw the dot by drawing a circle with diameter 5
// at the current origin (0,0) position, which has
// been manipulated by the two calls to translate() above
noStroke();
fill(255);
circle(0, 0, 5);

// Draw another dot orbiting the first dot
// You can adjust the following value to change the speed of the second dot
let angle = frameCount * width / 80;
let orbitDistance = 5; // Distance from the first dot
let orbitX = cos(angle) * orbitDistance;
let orbitY = sin(angle) * orbitDistance;

translate(orbitX, orbitY);
fill(255, 0, 0); // Make the second dot red
circle(0, 0, 3); // And slightly smaller than the first dot
}
```

Part 4: Transform a custom shape

Objective:

We are now going to transform a custom shape created from other shapes: an ellipse, rectangles and lines. The custom shape is based on the Bauhaus emblem designed by Oskar Schlemmer for the Staatliches Bauhaus. Bauhaus was a German school in the early 20th century founded on becoming a nexus for all forms of art. For more information, see:

<https://en.wikipedia.org/wiki/Bauhaus>  <https://en.wikipedia.org/wiki/Bauhaus>). You can find a version of the original emblem here: [File:Bauhaus-Signet.svg - Wikimedia Commons](https://commons.wikimedia.org/wiki/File:Bauhaus-Signet.svg)  <https://commons.wikimedia.org/wiki/File:Bauhaus-Signet.svg>).

Challenge:

We need a way to move, rotate and scale a shape or design that is made up of other shapes. Intuitively we might think that we need to manipulate the co-ordinates of each individual shape, however, since transformations in p5.js operate on the co-ordinate system, we can simply apply the transformations first, draw each shape in turn, and the entire custom shape will appear transformed as desired.

Solution:

We first create a class representing the Bauhaus head emblem called `Head`. The class will have a single method called `display()` which is used to draw the emblem. The first thing that happens inside the `display()` method is the `push()` function is called. The `push()` function preserves or saves any transformations and style settings. The fill colour is set, then the `translate()`, `rotate()` and `scale()` transforms are performed, in that order. The various shapes that make up the emblem are then drawn. Finally, `pop()` is called, which restores any transformations and style settings that were saved earlier.

Now that we have a `Head` class, we instantiate a single `head` object in the `setup()` function. And each frame (in `draw()`) we call the head object's `display()` method, passing in a value for the x-position that increases over time but will reset to 0 when it reaches the right side of the canvas (using `frameCount % width`). We also pass in a fixed value for the y-position, centring the head vertically, and we pass in values for rotation and scale that increase over time, again based on the `frameCount`. Lastly we pass in a fixed value for the colour.

Implementation:

```
let head = null;

function setup() {
  createCanvas(500, 500);
  angleMode(DEGREES);

  // Instantiate a single head object
  head = new Head();
}

function draw() {
  background(225);

  // Draw the head so that it:
  // - moves from left to right across the canvas,
  // - is centred vertically,
  // - and rotates and grows over time.
  head.display(
    (frameCount % width),
    height / 2,
    frameCount,
    (frameCount % width) / width * 200,
    color(245));
}

// Bauhaus Head Class
```

```

// The class has no constructor and a single display() method
// used to draw the Bauhaus emblem
class Head {
  display(x, y, angle, diameter, colour) {
    // The push() function saves any transformations and style settings
    // that might proceed a call to this method
    push();

    fill(colour);

    // Perform all transforms in the order: translate, rotate and scale
    translate(x, y);
    rotate(angle);
    scale(diameter);

    // Circular outline
    strokeWeight(0.01);
    ellipse(0, 0, 1);

    // Style for head
    fill(20);
    stroke(20);

    // Eye
    rect(-0.17, -0.33, 0.15, 0.15);
    line(-0.26, -0.33, -0.17, -0.33);
    line(-0.02, -0.18, -0.02, -0.09);

    // Nose
    rect(0.11, -0.48, 0.01, 0.59);
    line(0.02, 0.11, 0.11, 0.11);

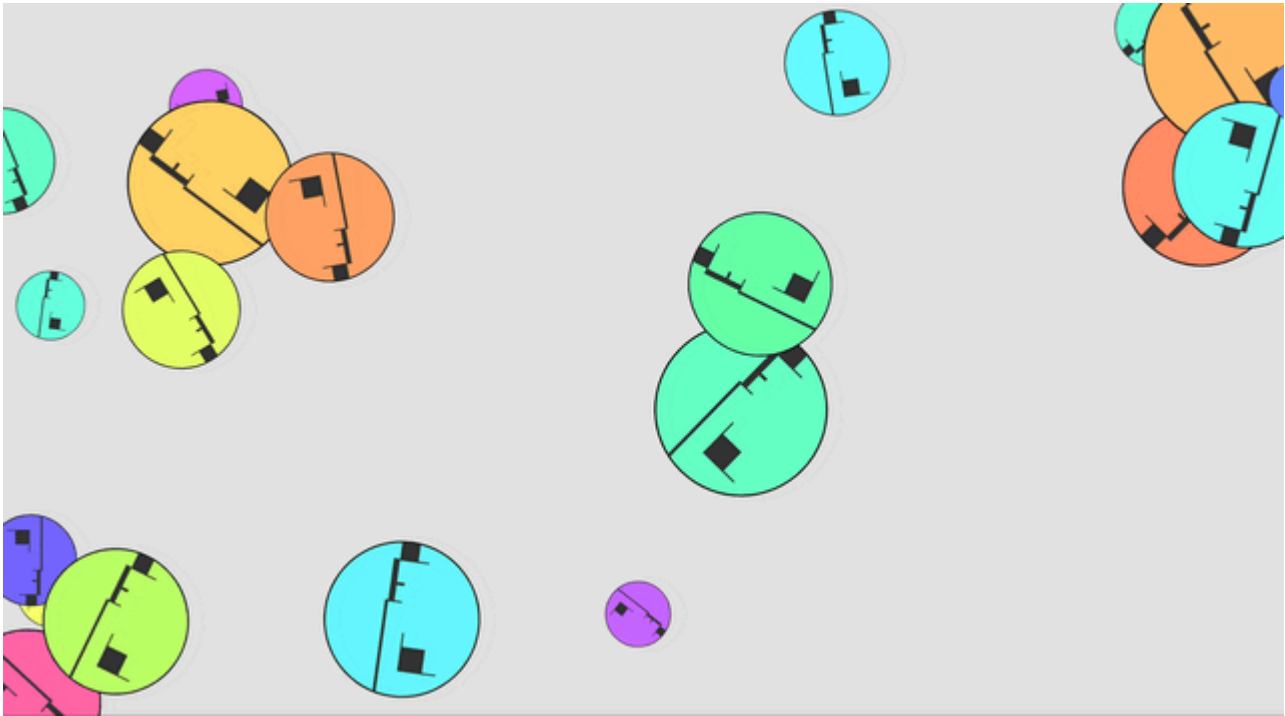
    // Mouth
    rect(0.07, 0.11, 0.03, 0.27);
    rect(0.02, 0.22, 0.04, 0.01);

    // Chin
    rect(-0.05, 0.38, 0.11, 0.11);
    line(-0.12, 0.38, -0.05, 0.38);

    // The pop() function restores any transformations and style settings
    // that were saved by the earlier call to push()
    pop();
  }
}

```

Part 5: Multiple custom shapes



Objective:

We will now modify the sketch to draw multiple Bauhaus heads of varying colours, and make them appear to roll across the canvas by adjusting their position and angle over time.

Challenge:

In the previous example we drew a single head by creating a single instance of the `Head` class and calling its `display()` method. We now need multiple instances of the `Head` class. In addition, we need a way to keep track of each head's position and angle as they move across the canvas.

Solution:

The easiest way to keep track of each head's position and angle as it moves across the canvas, is to encapsulate those values in the class. We will need to change the `Head` class as follows:

- Add a constructor to the `Head` class to initialise the position (x and y), angle, diameter and colour.
- Adjust the `display()` method to utilise the variables that are now encapsulated in the class.
- Add an `update()` method to update the x-position and angle each frame.

In addition to these changes, we need to initialise the head objects using random values so they vary in terms of starting position, starting angle, starting size and colour. We do this in a separate `initialiseHeads()` function which we call from `setup()`.

With all those changes now implemented, we simply need to call all the head objects' `display()` and `update()` methods every frame to animate and display each one correctly.

Implementation:

```

// Let's create an empty array to hold the multiple heads
let heads = [];
const numHeads = 20; // Adjust to change the number of heads displayed
const stepSize = 1; // Adjust to change the speed of animation

// We initialise the heads using a function
// Each head will have a random starting position,
// random starting angle, starting size and colour
// Note: the colour defined assumes HSB colour mode
function initialiseHeads() {
  // Clear the heads array
  // Note: only use this approach if nothing else is referencing the array
  heads = [];

  // Instantiate all the head objects with random values as outlined above
  for (let i = 0; i < numHeads; i++) {
    let head = new Head(
      random(width),
      random(height),
      random(360),
      random(50, 150),
      color(random(360), 60, 100))
    heads.push(head); // Add each head object to the array
  }
}

function windowResized() {
  resizeCanvas(windowWidth, windowHeight);
}

function setup() {
  createCanvas(windowWidth, windowHeight);
  angleMode(DEGREES);
  colorMode(HSB); // Use HSB colour mode
  initialiseHeads();
}

function draw() {
  background(0, 0, 88); // HSB(0, 0, 8) is approximately RGB(225, 225, 225)

  // Each frame we update the position and angle of each head
  // and display it
  for (const head of heads) {
    head.update();
    head.display();
  }
}

// Bauhaus Head Class
// The class has a constructor and two methods:
// - display() is used to draw the Bauhaus emblem
// - update() is used to update key variables each frame
class Head {
  // The constructor is used to set the initial
  // position, angle, diameter and colour
  constructor(x, y, angle, diameter, colour) {
    this.x = x;
    this.y = y;
    this.angle = angle;
    this.diameter = diameter;
    this.colour = colour;
  }

  display() {
    // The push() function saves any transformations and style settings
    // that might proceed a call to this method
    push();

    fill(this.colour);

    // Perform all transforms in the order: translate, rotate and scale
    translate(this.x, this.y);
  }
}

```

```

rotate(this.angle);
scale(this.diameter);

// Circular outline
strokeWeight(0.01);
ellipse(0, 0, 1);

// Style for head
fill(20);
stroke(20);

// Eye
rect(-0.17, -0.33, 0.15, 0.15);
line(-0.26, -0.33, -0.17, -0.33);
line(-0.02, -0.18, -0.02, -0.09);

// Nose
rect(0.11, -0.48, 0.01, 0.59);
line(0.02, 0.11, 0.11, 0.11);

// Mouth
rect(0.07, 0.11, 0.03, 0.27);
rect(0.02, 0.22, 0.04, 0.01);

// Chin
rect(-0.05, 0.38, 0.11, 0.11);
line(-0.12, 0.38, -0.05, 0.38);

// The pop() function restores any transformations and style settings
// that were saved by the earlier call to push()
pop();
}

// The update() method is intended to be called every frame
// The purpose of the update() method is to move and rotate the heads
// across the canvas from left to right.
// Note: the y-position, diameter and colour stay the same each frame
update() {
  // Adjust the x-position each frame
  this.x += stepSize;

  // Check to see if the head has moved past the right side of the canvas
  // and if it has, reset its position to the left side of the canvas
  if (this.x - this.diameter / 2 > width) {
    this.x = - this.diameter / 2;
  }

  // Adjust the angle each frame
  this.angle += stepSize;
}
}

```

Inspiration for the week

This week's inspiration is from teamLab, an international art collective formed in Tokyo, Japan in 2001. teamLab specialise in producing large-scale interactive artworks that feature at exhibitions and art galleries. The collective comprises an interdisciplinary team of artists, programmers, engineers, animators, mathematicians and architects.

If you take a close look at a video of their artwork, [Proliferating Immense Life \(YouTube\)](https://www.youtube.com/watch?v=s6Z_D-BPhs4)  (https://www.youtube.com/watch?v=s6Z_D-BPhs4), you should be able to observe elements that were covered in this week's tutorial: transformations (translate, rotate, scale), easing and timed events.

You can explore more of teamLab's artworks here: <https://www.teamlab.art/art> 
(<https://www.teamlab.art/art>)

Copyright © The University of Sydney. Unless otherwise indicated, 3rd party material has been reproduced and communicated to you by or on behalf of the University of Sydney in accordance with section 113P of the Copyright Act 1968 (Act). The material in this communication may be subject to copyright under the Act. Any further reproduction or communication of this material by you may be the subject of copyright protection under the Act. Do not remove this notice.

Live streamed classes in this unit may be recorded to enable students to review the content. If you have concerns about this, please visit our [student guide \(https://canvas.sydney.edu.au/courses/4901/pages/zoom\)](https://canvas.sydney.edu.au/courses/4901/pages/zoom) and contact the unit coordinator.

[Privacy Statement \(https://www.sydney.edu.au/privacy-statement.html\)](https://www.sydney.edu.au/privacy-statement.html)